AgriLink

Input & Procurement Management Module

Complete Source-Code Explanation & Architecture Guide

Java 21 | Spring Boot 4.0.6 | Spring Data JPA | MySQL | Lombok | SLF4J / Logback | JUnit 5

Prepared for Interview & Presentation

Group: **com.cts.agrilink** | Artifact: **agrilink**

REST base path: `/agriLink/procurementManagement` | Port: **8088**

Table of Contents

1. Project Overview & Purpose

AgriLink is a backend system that digitises how farmers obtain agricultural inputs (seeds, fertilisers, pesticides, equipment) from government/cooperative procurement centres. This document explains the **Input & Procurement Management** module, which exposes a REST API to manage two things:

- **The Input Catalog** — the master list of available agri-inputs, their prices, stock and status.
- **Input Requests** — orders raised by farmers that move through an approval → dispatch → delivery workflow.

> **One-line elevator pitch (for the interviewer):** "AgriLink is a Spring Boot REST service that manages an agricultural input catalog and the full lifecycle of farmer procurement requests — from creation, through approval and dispatch, to delivery — backed by MySQL via Spring Data JPA, with validation, centralised exception handling, and SLF4J logging."

The two functional areas

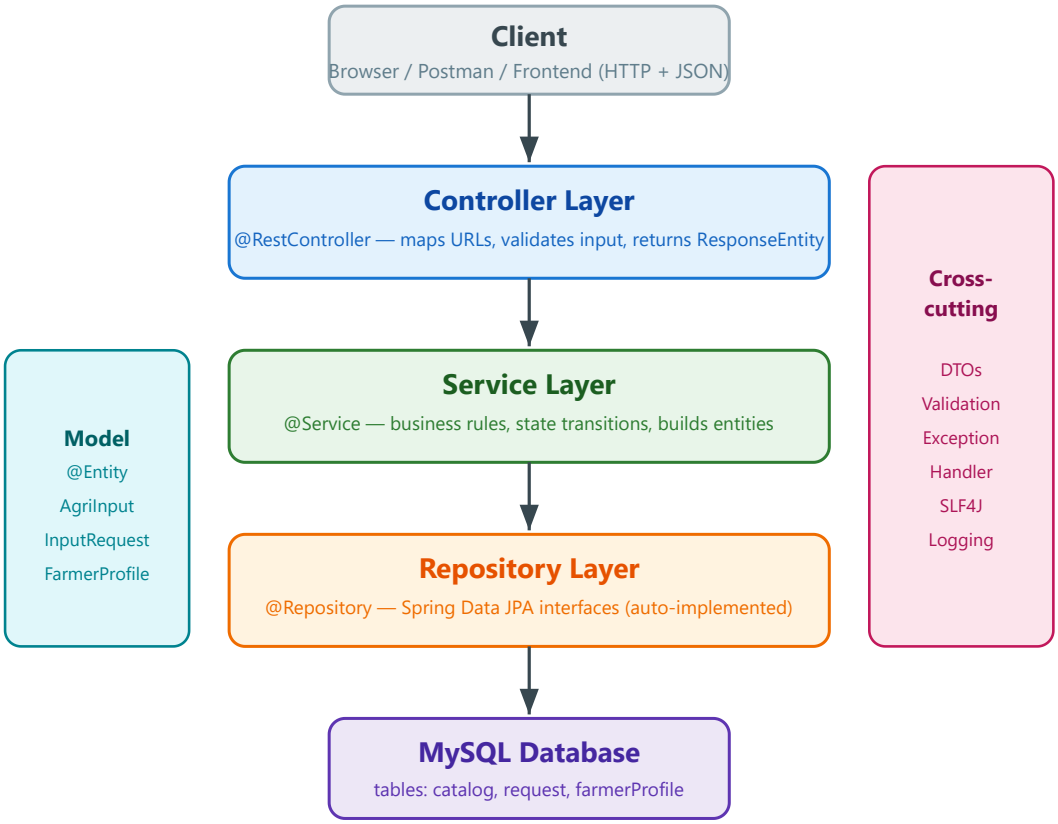
Area	Entity	Base URL	What it does
Catalog management	`AgriInput`	`/inputs`	CRUD on the catalog of inputs; query by category/status; update stock levels.
Procurement workflow	`InputRequest`	`/input-requests`	Farmers submit requests; staff approve, dispatch, deliver or cancel them; query by farmer/status/centre.

2. Technology Stack

Layer / Concern	Technology	Why it is used
Language	Java 21	Modern LTS Java; records, switch expressions, etc.
Framework	Spring Boot 4.0.6	Auto-configuration, embedded Tomcat, dependency injection.
Web	Spring Web MVC	Builds the REST controllers (`@RestController`).
Persistence	Spring Data JPA + Hibernate	Maps Java objects to MySQL tables; generates queries from method names.
Database	MySQL	Relational store for catalog and requests.
Validation	Jakarta Bean Validation	`@NotNull` , `@Positive` etc. validate incoming JSON.
Boilerplate	Lombok	Generates getters/setters/builders/loggers at compile time.
Logging	SLF4J facade over Logback	Leveled, configurable logging to console + rolling file.
Security util	spring-security-crypto	BCrypt password hashing only (no web security here).
Testing	JUnit 5, Mockito, H2	Unit + slice tests; H2 in-memory DB for repository tests.

3. Layered Architecture

The module follows the classic **Spring layered (n-tier) architecture**. Each layer has one responsibility and only talks to the layer directly beneath it. This separation is the single most important thing to explain in an interview.



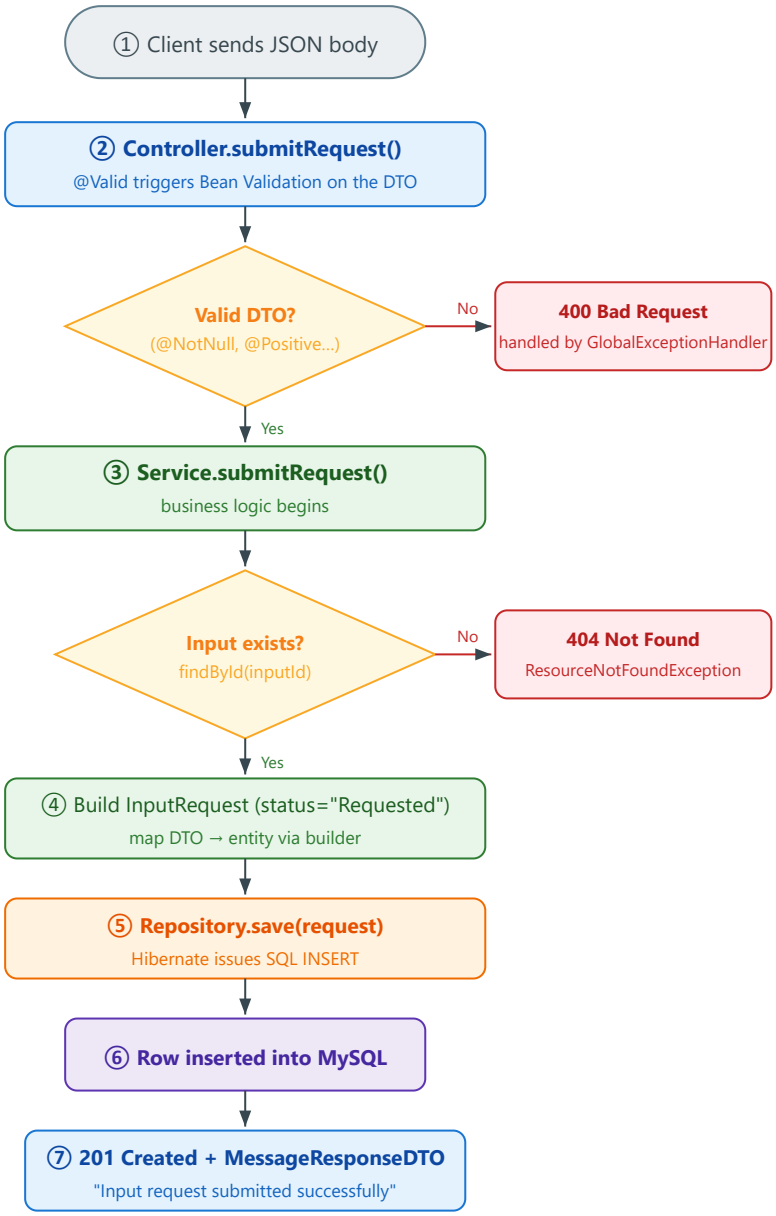
Requests flow DOWN; data/responses flow UP the same path.

Responsibility of each layer

Layer	Annotation	Responsibility	Must NOT do
Controller	`@RestController`	Receive HTTP, validate body, delegate, shape HTTP response & status code.	No business rules, no DB calls.
Service	`@Service`	Business logic: status checks, state changes, mapping DTO→entity.	No HTTP concepts, no SQL.
Repository	`@Repository`	Persistence: save/find/delete via Spring Data JPA.	No business rules.
Model	`@Entity`	Maps a Java class to a DB table (the data shape).	No logic.
DTO	(plain + validation)	Carry & validate data crossing the API boundary.	Never persisted directly.

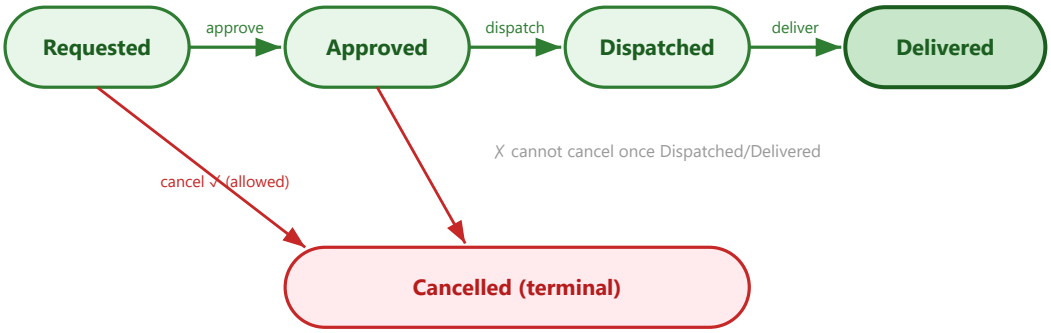
4. End-to-End Request Flowchart

This traces a single concrete call — `POST /input-requests` (a farmer submitting a request) — through every layer, including the validation and exception branches. This is the diagram to draw on a whiteboard when asked "walk me through what happens when a request comes in".



5. Request Lifecycle State Machine

An `InputRequest` has a `status` field that may only move along legal transitions. The service layer enforces these rules — any illegal jump throws a `StateConflictException` (HTTP 409). **This state machine is the heart of the module's business logic.**



Action	Allowed only from	Moves to	If violated
`approve`	Requested	Approved	409 Conflict (StateConflictException)
`dispatch`	Approved	Dispatched	
`deliver`	Dispatched	Delivered	
`cancel`	Requested or Approved	Cancelled	

6. Entry Point — `AgrilinkApplication.java`

Every Spring Boot application starts from one class with a `main` method. This is the "ignition key" of the whole service.

```java
package com.cts.agrilink;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AgrilinkApplication {
    public static void main(String[] args) {
        SpringApplication.run(AgrilinkApplication.class, args);
    }
}
```

Line	Code	Explanation
1	`package com.cts.agrilink;`	Declares the root package. Being at the root matters: Spring Boot's component scan starts here and scans **all sub-packages** — so every `@Controller`, `@Service`, `@Repository` below it is auto-discovered.
3	`import …SpringApplication;`	The bootstrap helper class that creates and runs the Spring application context.
4	`import …SpringBootApplication;`	Imports the master annotation used on line 6.
6	`@SpringBootApplication`	The key annotation. It is a **meta-annotation** bundling three: `@Configuration` (this class can define beans), `@EnableAutoConfiguration` (auto-configure Tomcat, JPA, DataSource from the classpath + properties), and `@ComponentScan` (find all components in this package and below).
7	`public class AgrilinkApplication`	The application class itself. No body needed beyond `main`.
9	`public static void main(String[] args)`	Standard Java entry point — the JVM calls this first.
11	`SpringApplication.run(AgrilinkApplication.class, args);`	Boots everything: creates the **ApplicationContext**, performs component scan, runs auto-configuration, starts the embedded **Tomcat** server on port 8088, and wires all beans together via dependency injection.

7. Configuration

7.1 `application.properties`

This file externalises all environment/config so no settings are hard-coded in Java.

```
spring.application.name=agrilink
spring.datasource.url = jdbc:mysql://localhost:3306/agrilink
spring.datasource.username = root
spring.datasource.password =root

spring.jpa.show = true
spring.jpa.generate-ddl = true
spring.jpa.hibernate.ddl.auto = update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
server.port = 8088

# Logging (SLF4J over Logback)
logging.level.root=INFO
logging.level.com.cts.agrilink=DEBUG
logging.file.name=logs/agrilink.log
```

Property	Meaning
`spring.application.name`	Logical name of the app (appears in logs/monitoring).
`spring.datasource.url`	JDBC URL — connect to the MySQL `agrilink` schema on localhost:3306.
`spring.datasource.username` / `password`	DB credentials used by the HikariCP connection pool.
`spring.jpa.hibernate.ddl.auto = update`	On startup Hibernate compares entities to tables and **creates/alters** tables to match (never drops). This is why the `catalog`, `request`, `farmerProfile` tables appear automatically.
`spring.jpa…dialect = MySQLDialect`	Tells Hibernate to generate MySQL-flavoured SQL.
`server.port = 8088`	Embedded Tomcat listens on 8088 instead of the default 8080.
`logging.level.com.cts.agrilink=DEBUG`	Our own packages log at DEBUG (verbose); everything else stays at INFO.
`logging.file.name`	Also write logs to a rolling file under `logs/`.

> **Interview note:** in production you would set `ddl.auto=validate` (never auto-modify schema) and move credentials to environment variables / a secrets manager, not plain text.

7.2 `pom.xml` — key dependencies

Dependency	Purpose
`spring-boot-starter-data-jpa`	Brings in Spring Data JPA + Hibernate — the persistence layer.
`spring-boot-starter-webmvc`	Brings in Spring MVC + embedded Tomcat — lets us build REST controllers.
`spring-boot-starter-validation`	Enables Jakarta Bean Validation (the `@NotNull` / `@Positive` annotations).
`mysql-connector-j`	The MySQL JDBC driver (runtime scope).
`lombok`	Compile-time generation of getters/setters/builders/ `log`.
`spring-security-crypto`	BCrypt hashing utility only (no full web security).
`h2` (test)	In-memory database used by repository slice tests.

8. Model / Entity Layer

Entities are plain Java classes annotated so JPA/Hibernate can map each one to a database table. One entity instance = one table row.

8.1 `AgriInput.java` — the catalog table

```
@Entity
@Table(name = "catalog")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class AgriInput {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "inputId")
    private Long inputId;
    @Column(name="name")        private String name;
    @Column(name="category")    private String category;
    @Column(name="unit")        private String unit;
    @Column(name="pricePerUnit")    private Double pricePerUnit;
    @Column(name="subsidisedPrice") private Double subsidisedPrice;
    @Column(name="availableStock")  private Integer availableStock;
    @Column(name="status")      private String status;
}
```

Line / Element	Explanation
`@Entity`	Marks the class as a JPA entity — Hibernate will manage it and map it to a table.
`@Table(name="catalog")`	Maps this entity to the DB table named `catalog` (instead of the default class name).
`@Data`	Lombok: generates getters, setters, `toString()`, `equals()` / `hashCode()` for all fields.
`@NoArgsConstructor`	Lombok: generates an empty constructor — **required by JPA** to instantiate entities via reflection.
`@AllArgsConstructor`	Lombok: constructor with every field as a parameter.
`@Builder`	Lombok: enables the fluent builder pattern, e.g. `AgriInput.builder().name("Urea").build()` — used by the service.
`@Id`	Declares `inputId` as the primary key.
`@GeneratedValue(IDENTITY)`	The DB auto-increments the PK (MySQL `AUTO_INCREMENT`); we never set it manually.
`@Column(name="…")`	Maps each field to a named column. The `status` field holds "Available"/"OutOfStock"; `category` holds Seed/Fertiliser/Pesticide/Equipment.

8.2 `InputRequest.java` — the request table

Same annotation pattern (`@Entity @Table(name="request") @Data …`), but with more fields because it tracks a workflow. The important columns:

Field	Type	Role in the workflow
`requestId`	Long (PK)	Auto-generated identity.
`farmerId, inputId`	Long	Who requested, and which catalog item (logical references).
`quantityRequested`	Integer	How many units.
`requestDate`	LocalDate	When the request was raised.

`assignedCentreId`	Long	Procurement centre handling it.
`actualPrice`	Double	Final price set at approval.
`status`	String	**The state-machine field**: Requested → Approved → Dispatched → Delivered / Cancelled.
`approvedBy, dispatchedBy, receivedBy`	String	Audit trail — who performed each step.
`dispatchDate, deliveredDate`	LocalDate	Timestamps for dispatch and delivery.
`cancellationReason`	String	Why a request was cancelled.

8.3 `FarmerProfile.java`

Maps to table `farmerProfile`; stores farmer demographics (name, DOB, village, district, bank account, status). Note `userId` is a plain `Long` — a **logical** link to the user-management module's user table, not a JPA foreign key, which keeps this module loosely coupled.

9. DTO Layer (Data Transfer Objects)

A **DTO** is a simple object that carries data across the API boundary. We never expose entities directly to clients — DTOs let us (a) accept exactly the fields a request needs, (b) validate them, and (c) keep the database shape decoupled from the API contract.

> **Why all DTOs share these three Lombok annotations**
> `@Data` → getters/setters/toString; `@NoArgsConstructor` → empty constructor so Jackson can deserialize incoming JSON; `@AllArgsConstructor` → full constructor for easy creation in tests.

9.1 Input DTOs — example `InputProcurementRequestDTO` (submit a request)

```
@Data @NoArgsConstructor @AllArgsConstructor
public class InputProcurementRequestDTO {
    @NotNull(message="Farmer ID is required")      private Long farmerId;
    @NotNull(message="Input ID is required")       private Long inputId;
    @NotNull @Positive(message="Quantity must be positive") private Integer quantityRequested;
    @NotNull(message="Request date is required")   private LocalDate requestDate;
    @NotNull(message="Assigned centre ID is required") private Long assignedCentreId;
    private Double actualPrice;
}
```

Annotation	Explanation
`@NotNull`	Field must be present in the JSON and not null; otherwise validation fails with the given message.
`@Positive`	The number must be > 0 (quantity cannot be zero or negative).
`message="…"`	The exact error text returned to the client (collected by the GlobalExceptionHandler into a 400 response).
`actualPrice (no annotation)`	Optional — may be left out at submit time; it is finalised at approval.

9.2 Validation annotations used across all DTOs

Annotation	Rule	Used in
`@NotNull`	Value must not be null.	IDs, dates, prices, stock.
`@NotBlank`	String must not be null or empty/whitespace.	name, category, unit, status, approvedBy, dispatchedBy, receivedBy, reason.
`@Positive`	Number > 0.	quantity, pricePerUnit.
`@PositiveOrZero`	Number ≥ 0.	availableStock (0 = out of stock is valid).

9.3 The full DTO catalogue

DTO	Direction	Carries / validates
`AgriInputRequestDTO`	in	Create a catalog item: name, category, unit, pricePerUnit (+ optional subsidised), availableStock, status.
`UpdateInputDTO`	in	Edit a catalog item: name, price, subsidised, stock, status.
`UpdateStockDTO`	in	Stock-only update: just `availableStock` (≥ 0).
`InputProcurementRequestDTO`	in	Submit a farmer request (see 9.1).
`UpdateInputRequestDTO`	in	Edit a pending request: quantity, centre, price — all optional (partial update).
`ApproveRequestDTO`	in	Approve: assignedCentreId, actualPrice, approvedBy (all required).

`DispatchRequestDTO`	in	Dispatch: dispatchedBy, dispatchDate.
`DeliverRequestDTO`	in	Deliver: deliveredDate, receivedBy.
`CancelRequestDTO`	in	Cancel: reason (required).
`MessageResponseDTO`	**out**	The standard success/error response — a single `message` string.

Note: `UpdateInputRequestDTO` fields are intentionally **not** `@NotNull` — it supports partial updates, and the service only applies the fields that were actually provided (null-checks each one).

10. Repository Layer

Repositories are **interfaces** — we write no implementation. Spring Data JPA generates the implementation at runtime, including SQL derived from the method names. This is the biggest "magic" point to explain in an interview.

10.1 `AgriInputRepository.java`

```
@Repository
public interface AgriInputRepository extends JpaRepository<AgriInput, Long> {
    List<AgriInput> findByCategoryIgnoreCase(String category);
    List<AgriInput> findByStatusIgnoreCase(String status);
}
```

Line	Code	Explanation
1	`@Repository`	Marks it as a persistence component (also translates DB exceptions into Spring's `DataAccessException`).
2	`extends JpaRepository<AgriInput, Long>`	Inherits ready-made CRUD methods for the `AgriInput` entity whose primary key is a `Long`: `save()`, `findById()`, `findAll()`, `deleteById()`, `existsById()`, `count()` — all for free.
3	`findByCategoryIgnoreCase(String)`	**Derived query.** Spring parses the name and generates `WHERE UPPER(category)=UPPER(?)`. "IgnoreCase" makes the match case-insensitive. No SQL written by hand.
4	`findByStatusIgnoreCase(String)`	Same idea, filtering on the `status` column.

10.2 `InputRequestRepository.java`

```
@Repository
public interface InputRequestRepository extends JpaRepository<InputRequest, Long> {
    List<InputRequest> findByFarmerId(Long farmerId);
    List<InputRequest> findByStatusIgnoreCase(String status);
    List<InputRequest> findByAssignedCentreId(Long centreId);
}
```

Method	Generated query
`findByFarmerId(Long)`	`SELECT * FROM request WHERE farmerId = ?`
`findByStatusIgnoreCase(String)`	`… WHERE UPPER(status) = UPPER(?)`
`findByAssignedCentreId(Long)`	`… WHERE assignedCentreId = ?`

> **Interview gold:** "Spring Data JPA uses *derived query methods* — it parses the method name into a JPQL/SQL query at startup, so I get type-safe finders without writing or maintaining SQL. For anything complex I could drop to `@Query` with JPQL or native SQL."

11. Service Layer (Business Logic)

This is where the real work happens — the rules that make the app correct. Controllers just delegate here.

11.1 Class declaration & dependency injection (common to both services)

```
@Service
@Slf4j
@RequiredArgsConstructor
public class InputRequestService {
    private final InputRequestRepository inputRequestRepository;
    private final AgriInputRepository agriInputRepository;
```

Element	Explanation
`@Service`	Marks this as a service bean so Spring creates one instance and injects it where needed.
`@Slf4j`	Lombok generates `private static final Logger log = LoggerFactory.getLogger(InputRequestService.class);` — gives us `log.info(...)` etc. for free (SLF4J facade).
`@RequiredArgsConstructor`	Lombok generates a constructor taking all `final` fields. Spring uses that constructor to **inject** the two repositories — this is **constructor injection**, the recommended DI style.
`private final …Repository`	The collaborators this service needs. `final` = set once at construction; guarantees they are never null.

11.2 A read method — `getRequestById`

```
public InputRequest getRequestById(Long requestId) {
    log.debug("Fetching input request with ID: {}", requestId);
    return inputRequestRepository.findById(requestId)
        .orElseThrow(() -> new ResourceNotFoundException(
            "Input request not found with ID: " + requestId));
}
```

Line	Explanation
log.debug("...{}", requestId) — logs at DEBUG with a parameterised placeholder; the string is only built if DEBUG is enabled (efficient).	
findById(requestId) returns an `Optional<InputRequest>` — it may or may not contain a value.	
.orElseThrow(...) — if the Optional is empty (no such row), throw `ResourceNotFoundException`, which the GlobalExceptionHandler turns into a clean **404**. Otherwise return the found entity. This null-safe pattern repeats in every "by ID" method.	

11.3 The state-transition method — `approveRequest` (the most important method)

```
public MessageResponseDTO approveRequest(Long requestId, ApproveRequestDTO dto) {
    InputRequest request = inputRequestRepository.findById(requestId)
        .orElseThrow(() -> new ResourceNotFoundException("… " + requestId));

    if (!"Requested".equals(request.getStatus())) {
        log.warn("Approve rejected for request ID {}: current status is '{}'",
                 requestId, request.getStatus());
        throw new StateConflictException(
            "Request cannot be approved. Current status: " + request.getStatus());
    }

    request.setStatus("Approved");
    request.setAssignedCentreId(dto.getAssignedCentreId());
    request.setActualPrice(dto.getActualPrice());
    request.setApprovedBy(dto.getApprovedBy());

    inputRequestRepository.save(request);
    log.info("Approved input request ID: {} by {}", requestId, dto.getApprovedBy());
    return new MessageResponseDTO("Input request approved successfully");
}
```

Step	Explanation
`findById…orElseThrow`	**Load** the request or 404 if it doesn't exist.
`if (!"Requested".equals(status))`	**Guard / state check.** Approval is only legal from the "Requested" state. Writing the literal first (`"Requested".equals(...)`) avoids a `NullPointerException` if status were null.
`log.warn(…)`	Records the rejected attempt at WARN level — useful for auditing misuse without it being an error.
`throw new StateConflictException`	Illegal transition → custom exception → GlobalExceptionHandler returns **409 Conflict**.
`request.setStatus("Approved") + setters`	**Apply the transition** and record the approval details (centre, price, who approved).
`repository.save(request)`	**Persist.** Because the entity already has an ID, Hibernate issues an `UPDATE`.
`log.info(…)`	Records the successful business event (audit trail).
`return new MessageResponseDTO(…)`	Returns a friendly success message to the controller.

dispatchRequest, deliverRequest, cancelRequest follow this exact 6-step shape — only the *allowed source state* and the *target state* differ (see the lifecycle table in §5). `cancelRequest` inverts the check: it forbids cancelling once the status is "Dispatched" or "Delivered".

11.4 The create method — `submitRequest`

```
public MessageResponseDTO submitRequest(InputProcurementRequestDTO dto) {
    AgriInput input = agriInputRepository.findById(dto.getInputId())
        .orElseThrow(() -> new ResourceNotFoundException("Input not found …"));

    InputRequest request = InputRequest.builder()
        .farmerId(dto.getFarmerId())
        .inputId(dto.getInputId())
        .quantityRequested(dto.getQuantityRequested())
        .requestDate(dto.getRequestDate())
        .assignedCentreId(dto.getAssignedCentreId())
        .actualPrice(dto.getActualPrice())
        .status("Requested")
        .build();

    inputRequestRepository.save(request);
    log.info("Submitted input request: farmerId={}, inputId={}, qty={}", …);
    return new MessageResponseDTO("Input request submitted successfully");
}
```

- **Referential check first:** verify the referenced catalog item exists (else 404) — you cannot request something that isn't in the catalog.

- **Builder pattern** (enabled by Lombok `@Builder`) maps the incoming DTO onto a new entity, fluently and readably.
- **status="Requested"** — every new request starts in the initial state of the lifecycle.
- **save()** → Hibernate `INSERT` (no ID yet → insert, not update).

11.5 `AgriInputService` — catalog logic

Same structure, simpler (no state machine). Notable method — validating an enum-like value:

```
public List<AgriInput> getInputsByCategory(String category) {
    List<String> valid = List.of("Seed","Fertiliser","Pesticide","Equipment");
    if (valid.stream().noneMatch(c -> c.equalsIgnoreCase(category))) {
        log.warn("Rejected request: invalid category '{}'", category);
        throw new IllegalArgumentException("Invalid category: " + category);
    }
    return agriInputRepository.findByCategoryIgnoreCase(category);
}
```

- Whitelists allowed categories; an unknown value throws `IllegalArgumentException` → **400 Bad Request**.
- `addInput / updateInput / updateStock / deleteInput` are straightforward CRUD: load-or-404, mutate, `save()`, return a message. `deleteInput` uses `existsById()` to 404 before deleting.

12. Controller Layer (REST API)

Controllers are the front door — they map HTTP verbs + URLs to Java methods, validate the body, delegate to the service, and wrap the result in a `ResponseEntity` with the correct status code.

12.1 Class declaration

```
@RestController
@RequestMapping("/agriLink/procurementManagement/input-requests")
@Slf4j
@RequiredArgsConstructor
public class InputRequestController {
    private final InputRequestService inputRequestService;
```

Element	Explanation
`@RestController`	`@Controller` + `@ResponseBody` — every method's return value is serialised straight to JSON (no view templates).
`@RequestMapping("…/input-requests")`	Base URL prefix for every endpoint in this class.
`@Slf4j`	Logger for request-level INFO logging.
`@RequiredArgsConstructor`	Injects the service via constructor.

12.2 A GET endpoint

```
@GetMapping("/{requestId}")
public ResponseEntity<InputRequest> getRequestById(@PathVariable Long requestId) {
    return ResponseEntity.ok(inputRequestService.getRequestById(requestId));
}
```

Element	Explanation
`@GetMapping("/{requestId}")`	Handles `GET /…/input-requests/5` . The `{requestId}` is a URL template variable.
`@PathVariable Long requestId`	Binds the `5` from the URL into the method parameter (auto-converted to Long).
`ResponseEntity.ok(…)`	Wraps the body with HTTP **200 OK**. `ResponseEntity` gives full control over status + headers + body.

12.3 A POST endpoint (with validation)

```
@PostMapping
public ResponseEntity<MessageResponseDTO> submitRequest(
        @Valid @RequestBody InputProcurementRequestDTO dto) {
    log.info("POST /input-requests - submit request for farmerId={}", dto.getFarmerId());
    return ResponseEntity.status(HttpStatus.CREATED)
                         .body(inputRequestService.submitRequest(dto));
}
```

Element	Explanation
`@PostMapping`	Handles `POST /…/input-requests` (create).
`@RequestBody`	Deserialises the incoming JSON body into the DTO (via Jackson).
`@Valid`	**Triggers Bean Validation** on the DTO. If any rule (e.g. `@NotNull`) fails, Spring throws `MethodArgumentNotValidException` **before** the method body runs →

	400.
`HttpStatus.CREATED`	Returns **201 Created** (the correct REST status for a successful creation) rather than 200.

12.4 The complete REST API surface

Catalog — base `/agriLink/procurementManagement/inputs`

Method	Path	Action	Success
GET	/	List all inputs	200
GET	/{inputId}	Get one input	200
GET	/category/{category}	Filter by category	200
GET	/status/{status}	Filter by status	200
POST	/	Add input	201
PUT	/{inputId}	Update input	200
PUT	/{inputId}/stock	Update stock only	200
DELETE	/{inputId}	Delete input	200

Requests — base `/agriLink/procurementManagement/input-requests`

Method	Path	Action	Success
GET	/	List all requests	200
GET	/{requestId}	Get one request	200
GET	/farmer/{farmerId}	By farmer	200
GET	/status/{status}	By status	200
GET	/centre/{centreId}	By centre	200
POST	/	Submit request	201
PUT	/{requestId}	Update request	200
PUT	/{requestId}/approve	Approve	200
PUT	/{requestId}/dispatch	Dispatch	200
PUT	/{requestId}/deliver	Deliver	200
PUT	/{requestId}/cancel	Cancel	200
DELETE	/{requestId}	Delete request	200

13. Exception Handling Layer

Instead of writing try/catch in every controller, all errors are handled in **one central place**. The service throws meaningful exceptions; this advice class translates each one into the right HTTP status.

13.1 Custom exceptions

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}
public class StateConflictException extends RuntimeException {
    public StateConflictException(String message) { super(message); }
}
```

- Both extend `RuntimeException` (unchecked) so they propagate without clutter in method signatures.
- They are **semantic**: their type alone tells the handler which HTTP status to map to.

13.2 `GlobalExceptionHandler`

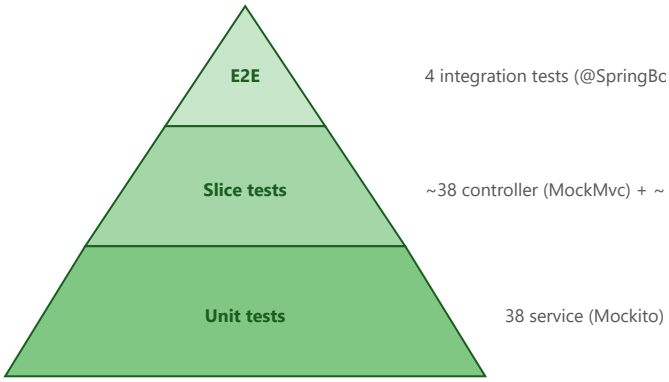
```
@RestControllerAdvice(basePackages = "com.cts.agrilink.inputAndProcurementMangement")
@Slf4j
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<MessageResponseDTO> handleNotFound(ResourceNotFoundException ex) {
        log.warn("Resource not found: {}", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
                             .body(new MessageResponseDTO(ex.getMessage()));
    }
    // … handlers for validation (400), StateConflict (409), IllegalArgument (400),
    // and a catch-all Exception (500) that logs the full stack trace.
}
```

Element	Explanation
`@RestControllerAdvice(basePackages=…)`	A global handler applied to all controllers **in this module's package only** — scoping it avoids clashing with the user-management module's own handler.
`@ExceptionHandler(X.class)`	"When any controller throws exception X, run this method instead of letting it crash to a default 500."
`log.warn / log.error`	Each handler logs the problem (WARN for client errors, ERROR + stack trace for unexpected faults).
`ResponseEntity.status(...).body(...)`	Returns a consistent JSON error shape (`MessageResponseDTO`) with the right status.

Exception thrown by service	Mapped HTTP status	Meaning to client
`ResourceNotFoundException`	404 Not Found	The ID you asked for doesn't exist.
`MethodArgumentNotValidException`	400 Bad Request	Your JSON failed validation (collects all field messages).
`IllegalArgumentException`	400 Bad Request	Invalid value (e.g. unknown category/status).
`StateConflictException`	409 Conflict	Illegal workflow transition.
`Exception` (catch-all)	500 Internal Server Error	Unexpected fault — logged with stack trace.

14. Test Suite Explanation (109 tests)

The module is covered by **109 automated tests** organised as a classic **test pyramid**: many fast unit tests at the bottom, fewer slice tests in the middle, and a small number of full end-to-end tests at the top. Each layer is tested in isolation so a failure points to exactly one place.



Test class	Type	Tooling	What it verifies	Tests
`AgriInputServiceTest`	Unit	Mockito	Catalog business logic, repos mocked	15
`InputRequestServiceTest`	Unit	Mockito	Workflow logic + state machine	23
`AgriInputControllerTest`	Web slice	MockMvc	Catalog HTTP/JSON + status codes	~18
`InputRequestControllerTest`	Web slice	MockMvc	Request HTTP/JSON + status codes	20
`AgriInputRepositoryTest`	Persistence	@DataJpaTest + H2	Catalog queries on real DB	~11
`InputRequestRepositoryTest`	Persistence	@DataJpaTest + H2	Derived queries on real DB	18
`InputProcurementIntegrationTest`	End-to-end	@SpringBootTest + H2	Full stack, nothing mocked	4
`AgrilinkApplicationTests`	Smoke	@SpringBootTest	Context loads	1

14.1 Unit tests (service layer) — Mockito

```
@ExtendWith(MockitoExtension.class)
class InputRequestServiceTest {
    @Mock  private InputRequestRepository inputRequestRepository;
    @Mock  private AgriInputRepository agriInputRepository;
    @InjectMocks private InputRequestService inputRequestService;
}
```

Element	Explanation
`@ExtendWith(MockitoExtension.class)`	Activates Mockito for JUnit 5 — processes the `@Mock` / `@InjectMocks` annotations.
`@Mock`	Creates a **fake** repository. It does nothing until told (stubbed). No real database involved → fast.
`@InjectMocks`	Creates the **real** service and injects the two fakes into it via the constructor.

The three Mockito verbs used throughout:

Pattern	Meaning
`when(repo.findById(1L)).thenReturn(Optional.of(req))`	**Stub** — "when this is called, return that".

`verify(repo).save(any())`	**Verify** — assert the service really called save.
`assertThatThrownBy(() -> …).isInstanceOf(StateConflictException.class)`	**Assert** (AssertJ) — the right exception is thrown.

The most important unit tests prove the **state machine** — both the legal move and the illegal one:

Test name	What it proves
`approveRequest_requestedStatus_approvesSuccessfully`	Requested → Approved works; sets approvedBy/price/centre.
`approveRequest_nonRequestedStatus_throwsStateConflictException`	Approving an already-Approved request is rejected (409).
`dispatchRequest_nonApprovedStatus_throwsStateConflictException`	Cannot dispatch before approval.
`cancelRequest_dispatchedStatus_throwsStateConflictException`	Cannot cancel once dispatched.
`updateRequest_nullFields_doesNotOverwriteExistingValues`	Partial update only changes provided fields.

14.2 Controller slice tests — MockMvc

```
mockMvc = MockMvcBuilders.standaloneSetup(inputRequestController)
        .setControllerAdvice(new GlobalExceptionHandler())
        .build();
```

- Spins up **only the controller** (no Tomcat, no DB) and wires in the real exception handler — so the HTTP contract (status code + JSON body) is verified, with the service mocked.
- `mockMvc.perform(post(...)).andExpect(status().isCreated())` simulates an HTTP request and asserts on the response.
- `jsonPath("$.message").value("…")` asserts the JSON body content.

Test	Asserts
`submitRequest_validDto_returns201`	Valid create → 201 Created.
`submitRequest_missingFarmerId_returns400`	`@Valid` rejects null farmerId → 400.
`getRequestById_nonExistingId_returns404`	ResourceNotFound → 404.
`approveRequest_invalidTransition_returns409`	StateConflict → 409.

14.3 Repository slice tests — `@DataJpaTest` + H2

```
@DataJpaTest
@ActiveProfiles("test")
class InputRequestRepositoryTest { … }
```

Element	Explanation
`@DataJpaTest`	Loads **only** the JPA layer (entities + repositories), not the whole app. Fast and focused.
`@ActiveProfiles("test")`	Activates `application-test.properties` → uses an **in-memory H2** database with `ddl-auto=create-drop`, so every run starts with a fresh schema. No MySQL needed.
`@BeforeEach setUp()`	Seeds 4 known rows (farmers 100–102 across centres 5/6, various statuses) so each query has a predictable expected result.

These confirm that the **derived query method names actually generate correct SQL**: `findByFarmerId(100L)` → 2 rows, `findByAssignedCentreId(5L)` → 3 rows, `findByStatusIgnoreCase("Delivered")` → the right row, and that `save()` both inserts (auto-generates the ID) and updates existing rows.

14.4 End-to-end integration test — `InputProcurementIntegrationTest` (new)

This is the top of the pyramid: **@SpringBootTest** loads the **entire** application context and drives requests through the **full real stack** — HTTP → Controller → Service → Repository → H2 — with **nothing mocked**. It proves the layers actually wire together.

```
@SpringBootTest
@ActiveProfiles("test")
class InputProcurementIntegrationTest {
    mockMvc = MockMvcBuilders.webAppContextSetup(webApplicationContext).build();
    // seed a catalog item, then exercise the workflow over real HTTP + DB
}
```

Test	What it does end-to-end
`fullLifecycle_submitApproveDispatchDeliver_persistsEachTransition`	Submits a request, then approves → dispatches → delivers it over HTTP, asserting after each step that the **status is really persisted** in the database; finally reads it back via the API and confirms `Delivered`.
`illegalTransition_dispatchWhileRequested_returns409_andStatusUnchanged`	Tries to dispatch a brand-new request (skipping approval) → 409, and verifies the row is **still "Requested"** — the state machine protected the data.
`submitForUnknownInput_returns404`	Submitting a request for a non-existent catalog item → 404, and **no row** is created.
`catalogCreatedViaApi_isQueryableViaApi`	Creates a catalog item through `POST /inputs`, then reads it back through `GET /inputs/category/…` — proving write-then-read works across the full stack.

> **Why this matters in an interview:** "My unit tests are fast and pinpoint logic bugs; my slice tests lock the HTTP contract and the SQL; and my end-to-end test proves the whole stack works together against a real (in-memory) database. That's the test pyramid — fast feedback at the bottom, high confidence at the top."

14.5 How to run the suite (regression testing)

```
mvnw -o test                      # run all 109 tests offline
mvnw -o test -Dtest=InputRequestServiceTest          # one class
mvnw -o test -Dtest=InputRequestServiceTest#approveRequest_requestedStatus_approvesSuccessfully
```

Every test must stay green after a change — that is regression testing. When a bug is fixed, a new test is added first so the bug can never silently return.

15. Interview Talking Points & Q&A

Architecture & design

Question	Strong answer
Walk me through your architecture.	Classic Spring layered design: Controller (HTTP) → Service (business logic) → Repository (persistence) → MySQL. DTOs cross the API boundary; entities map to tables; a global advice handles exceptions; SLF4J logs across layers.
Why separate DTOs from entities?	To decouple the API contract from the DB schema, validate input precisely, avoid over-exposing fields, and prevent accidental persistence of client data.
What is dependency injection here?	Constructor injection via Lombok's `@RequiredArgsConstructor` on `final` fields — Spring supplies the repositories. Improves testability (easy to mock) and immutability.
How do you enforce business rules?	In the service layer. e.g. a request can only be approved from "Requested"; illegal transitions throw `StateConflictException` → 409. The state machine lives in one place.

Persistence

Question	Strong answer
How do repositories work without code?	Spring Data JPA generates implementations at runtime; derived query methods like `findByStatusIgnoreCase` are parsed into SQL from the method name.
save() — insert or update?	If the entity has no ID (new) → INSERT; if it has an existing ID → UPDATE. Hibernate decides.
What does `ddl.auto=update` do?	Hibernate creates/alters tables to match entities on startup but never drops. In production I'd use `validate` + a migration tool (Flyway/Liquibase).

Validation, errors & logging

Question	Strong answer
How is input validated?	Jakarta Bean Validation annotations on DTOs (`@NotNull` , `@Positive` ...), triggered by `@Valid` in the controller; failures become 400 via the global handler.
Why centralised exception handling?	`@RestControllerAdvice` keeps controllers clean and guarantees consistent error responses and status codes across the whole module.
Why SLF4J not System.out?	SLF4J is a facade (swappable backend = Logback); supports levels, timestamps, file rotation, and per-package configuration — none of which println offers.
What would you improve?	Add service-layer `@Transactional` , replace String status with an `enum` , add pagination to list endpoints, use Flyway for schema, and add real foreign keys / auth.

> **Closing summary to memorise:** "AgriLink's procurement module is a layered Spring Boot REST service. Controllers handle HTTP and validation, services own the business rules including a strict request lifecycle state machine, and Spring Data JPA repositories persist to MySQL. Cross-cutting concerns — DTO validation, centralised exception-to-HTTP mapping, and SLF4J logging — keep the code clean, consistent, and production-minded."